Verificación formal
Introducción a SPIN

Sistemas Empotrados Avanzados
2021

Patricia Arroba
p.arroba@upm.es

VERIFICACIÓN FORMAL

1. Especificación:	**CUÁL** es el problema
2. Modelado:	**QUÉ** hace el sistema
3. Verificación:	**CUMPLE** la especificación/funcionalidad
4. Diseño/Impl:	**CÓMO** lo hace (Decisiones a tomar)
5. Verificación:	**CUMPLE** la especificación/funcionalidad
6. Análisis:	**CUMPLE** plazos

OBJETIVOS

- Introducción Spin y promela
  - Active proctype
  - Types
  - Statements
    - Asignaciones y expresiones
    - If, do y printf
- Ejemplo aplicación

VERIFICACIÓN FORMAL

- SPIN: Herramienta SW de verificación formal open-source
- Promela: Lenguaje de especificación de programas concurrentes
  - Modelo promela del sistema
- LTL: Linear Temporal Logic
  - Describe las relaciones causales del sistema

Especificaciones → LTL

Modelo sistema → FSM

promela → spin

VERIFICACIÓN FORMAL: Promela

- No es necesario aprender promela en profundidad
- Sólo comprender lo básico para utilizarlo para nuestra verificación
  - active proctype
  - Types
  - Statements
    - Asignaciones y expresiones
    - if, do, printf

VERIFICACIÓN FORMAL: Promela

- active proctype → declaración de un proceso

hello.pml

```
/* A "Hello World" Promela model for SPIN. */
active proctype Hello() {
printf("Hello process, my pid is: %d\n", pid);
} ...
```

```
bash$ spin hello.pml
```

VERIFICACIÓN FORMAL: Promela

- active proctype → declaración de un proceso

hello.pml

```
/* A "Hello World" Promela model for SPIN. */
active proctype Hello() {
printf("Hello process, my pid is: %d\n", pid);
```

active proctype:

Proceso que está ejecutándose desde el principio

bas

VERIFICACIÓN FORMAL: Promela

- types → Sólo int
  - int a; ← declaración
  - int a = 1; ← declaración + inicialización
  - a = 0; ← asignación
  - int a[7] = {0,1,2,3,4,5,6}; ← array de enteros

VERIFICACIÓN FORMAL: Promela

UNIVERSIDAD POLITÉCNICA DE MADRID

- Statements
  - El cuerpo de un proceso es una secuencia de statements
  - Un statement puede:
    - Ser ejecutable ← Se puede ejecutar inmediatamente
    - Estar bloqueado ← No se puede ejecutar (El proceso no puede avanzar)

  Depende del estado del sistema

VERIFICACIÓN FORMAL: Promela

- Statements
  - **Asignaciones:** siempre ejecutables
    - b = 5
  - **Expresiones:** ejecutables si se cumplen
    - 2 < 3 ← ejecutable siempre
    - 2 > 3 ← siempre se bloquea
    - b < 5 ← sólo ejecutable si se cumple la condición
    - b == 5
    - b != 5

VERIFICACIÓN FORMAL: Promela

- Secuencia de statements en un proceso
  - b < 5; b = a*2 ← **Igual que un if**

    Si b < 5 ejecuta b=a*2

    Si no se queda bloqueado y el proceso no puede avanzar

    (b podría cambiar desde otro proceso)
  - Las instrucciones **if** son estructuras distintas!!!
  - <u>Nota:</u> en promela **;** es un separador no un terminador

VERIFICACIÓN FORMAL: Promela

- Statements
  - **Instrucción if**

```
if
:: b=1
:: a=1
:: c=1
:: skip
fi
```

  - Explorar entorno
    - Todas las posibles excitaciones

if = Estructura no deterministas

Ejecuta una de ellas aleatoriamente (= que un choice)

VERIFICACIÓN FORMAL: Promela

UNIVERSIDAD POLITÉCNICA DE MADRID

- Statements
  - **Instrucción do**

```
do
:: b=1
:: a=1
:: c=1
:: skip
od
```

do: Estructura no determinista

Ejecuta una de ellas aleatoriamente <u>en bucle</u>
(= que if pero en bucle)

- Statements
  - **Instrucción printf**

```
int a = 7
printf (“%d \n”, a) ← siempre es ejecutable

int p[7] = { 1,1,1,0,2,2,2 }
printf ("%d%d%d%d%d%d%d\n", p[0], p[1], p[2], p[3], p[4], p[5], p[6])
```

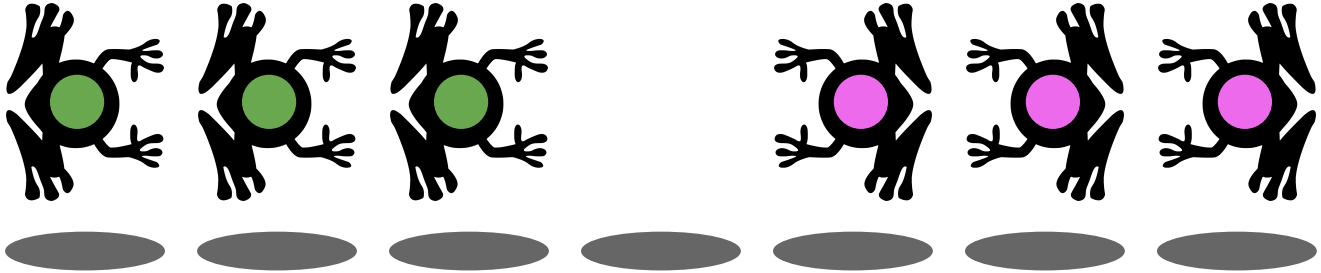
VERIFICACIÓN FORMAL: Spin

- Spin tiene dos modos:
  - Simular → Una posible traza de ejecución
    - `spin myfile.pml`
  - Verificar → Evalúa todas las posibles trazas de ejecución
    - `spin -a myfile.pml` → Genera un verificador (pan.c)
    - `gcc -o pan pan.c` → Compila el verificador
    - `./pan` → Ejecuta el verificador y genera contraejemplo si existe (myfile.pml.trail)
    - `spin -t myfile.pml` → Simula la traza que no verifica la propiedad

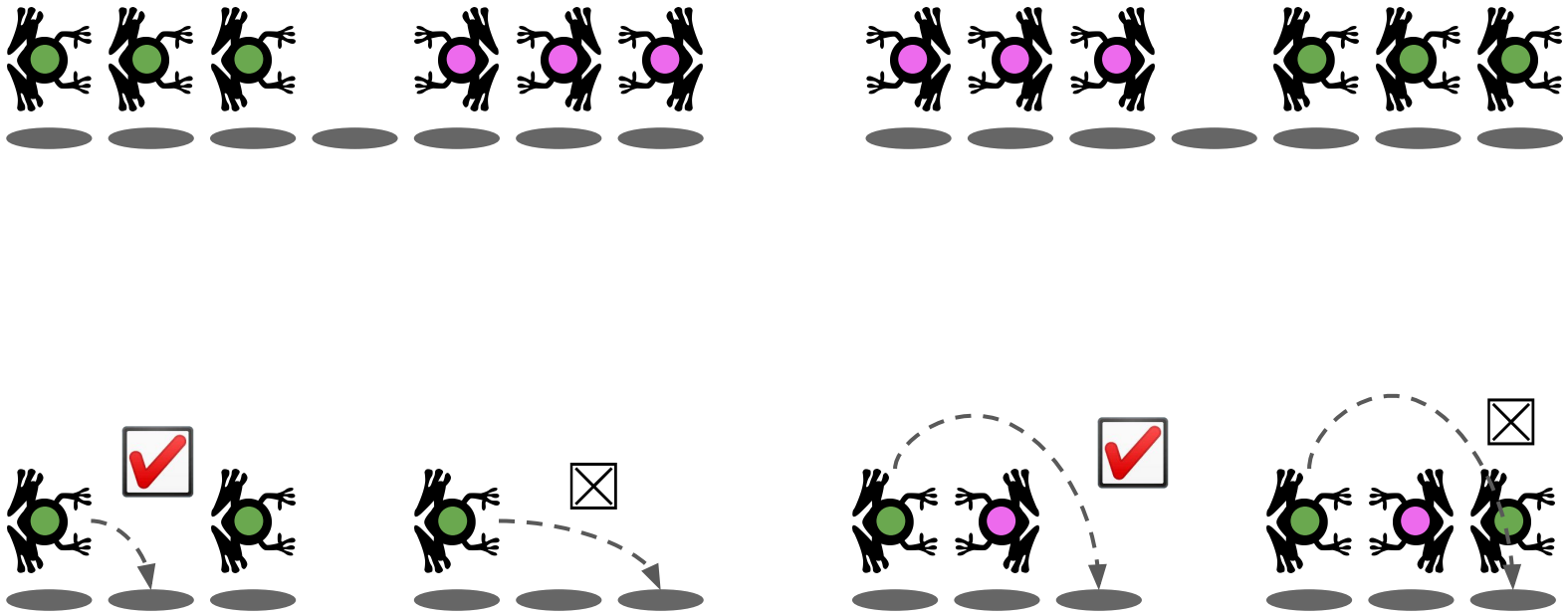
UNIVERSIDAD POLITÉCNICA DE MADRID

Ejemplo de problema de verificación

Problema:

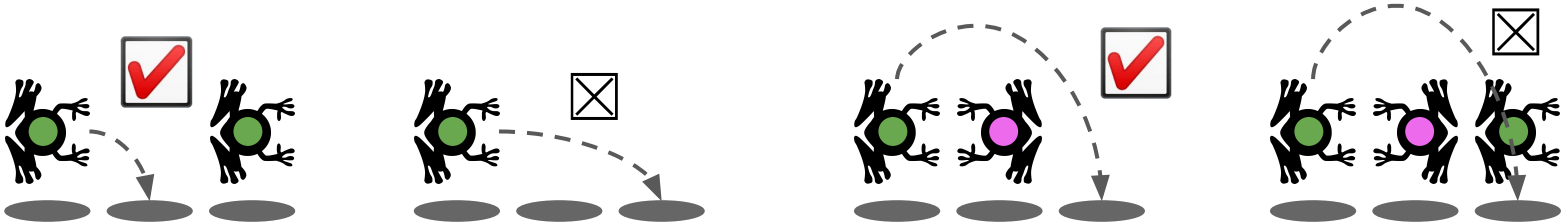


VERIFICACIÓN FORMAL



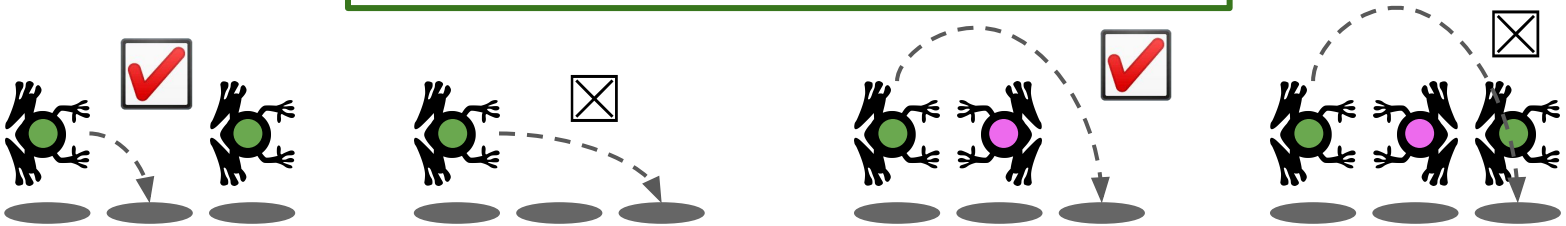
VERIFICACIÓN FORMAL

1º Simular con spin un modelo en promela de este escenario



VERIFICACIÓN FORMAL

2º Verificar con spin un modelo en promela de este escenario + especificación LTL



VERIFICACIÓN FORMAL: Spin

- Spin para verificar
  - Especificación LTL
    - ➢ Globalmente: G [] [ ]
    - ➢ Eventualmente en el futuro: F <> < >
  - La forma de verificar de spin es **encontrar contraejemplos** que no cumplan la especificación

¿Cómo conseguir que la verificación de spin me de la solución del problema?

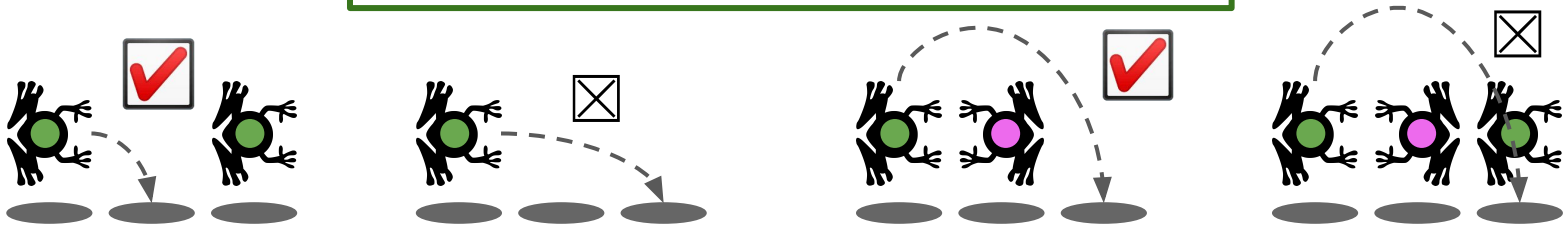
VERIFICACIÓN FORMAL: Spin

- Spin para verificar → Evalúa todas las posibles trazas de ejecución
  - Especificación LTL
    - ➢ Globalmente: G [] [ ]
    - ➢ Eventualmente en el futuro: F <> < >
  - La forma de verificar de spin es **encontrar contraejemplos** que no cumplan la especificación

Diciéndole en la especificación que nunca se encuentra la solución

VERIFICACIÓN FORMAL

2º Verificar con spin un modelo en promela de este escenario + especificación LTL



OBJETIVOS

- Introducción Spin y promela
  - Active proctype
  - Types
  - Statements
    - Asignaciones y expresiones
    - If, do y printf
- Ejemplo aplicación